Laboratório de Kubernetes

Da Máquina Local à Nuvem Híbrida

Bem-vindo ao laboratório!
Neste exercício, você vai sair do zero, criar um cluster Kubernetes na sua máquina, testar a auto-recuperação e, por fim, simular uma arquitetura de nuvem híbrida, enviando uma réplica da sua aplicação para o "datacenter" do professor.

A Aplicação: Vamos usar o Nginx, um servidor web leve e clássico. Ele será nossa cobaia para provar como o Kubernetes gerencia e move os contêineres.

Fase 1: O Básico (Primeiros Passos)

O objetivo aqui é instalar o Kubernetes, criar um Pod de forma imperativa e depois criar um Deployment (a forma correta).

1. Instalar o Kubernetes (k3s)

No seu terminal, execute o comando abaixo (ele baixa e instala o k3s, uma versão leve do Kubernetes):

```
curl -sfL https://get.k3s.io | sh -
```

Verifique se o nó (sua máquina) está pronto:

```
sudo kubectl get nodes
```

(Você deve ver o seu computador com o status `Ready`).

2. Criar um Pod solto (Modo Imperativo)

Vamos rodar um contêiner Nginx solto. **Atenção:** Pods soltos não têm auto-cura. Se ele morrer, o Kubernetes não faz nada.

```
sudo kubectl run meu-primeiro-pod --image=nginx:latest
sudo kubectl get pods
```

3. Criar um Deployment (Modo Declarativo)

Agora, vamos criar um gerenciador (`Deployment`) que garante que 3 réplicas do Nginx estejam sempre rodando.

```
sudo kubectl create deployment meu-app --image=nginx:latest --replicas=3
sudo kubectl get pods
```

(Você verá 3 pods com nomes aleatórios no final).

4. Expor a Aplicação para o Mundo (Service)

Vamos criar um Service do tipo `NodePort` para acessar a aplicação pelo navegador:

```
sudo kubectl expose deployment meu-app --type=NodePort --port=80
```

Descubra qual porta o Kubernetes abriu:

```
sudo kubectl get services
```

Procure a linha do `meu-app` na coluna `PORT(S)`, algo como `80:31234/TCP`.
Abra o navegador e acesse: `http://localhost:31234` (substitua `31234` pela porta que apareceu). Você verá a página de boas-vindas do Nginx!

Fase 2: Testando Conceito Local (Auto-cura)

Agora vamos provar que o Deployment mantém o estado desejado.

1. Simulando uma falha

O seu Deployment quer 3 réplicas. Vamos matar uma delas manualmente:

```
# Copie o nome de um dos pods listados
sudo kubectl get pods

# Delete o pod (substitua pelo nome que voce copiou)
sudo kubectl delete pod meu-app-<hash-aleatorio>
```

2. Observando a mágica

Rode o comando abaixo para ver os pods em tempo real:

```
sudo kubectl get pods -w
```

(Pressione `Ctrl+C` para sair).

O que aconteceu? O pod que você deletou entrou em `Terminating` e um novo pod foi criado automaticamente em segundos. O Kubernetes percebeu que você queria 3 e recriou o que faltava.

Fase 3: A Nuvem Híbrida (Simulando a AWS)

Agora, sua aplicação vai rodar não só na sua máquina, mas também no "datacenter do professor". A ideia é garantir que, se a sua máquina cair, a aplicação continue no ar.

1. Pegar seus dados de conexão

Para que o professor possa criar um "nó na nuvem" para você, passe os seguintes comandos e anote os resultados:

```
# 1. O IP da sua maquina na rede local:
hostname -I

# 2. O Token de seguranca do seu cluster (copie a string gigante):
sudo cat /var/lib/rancher/k3s/server/node-token
```

2. Contratar a Nuvem

Abra o navegador e acesse o portal do professor: `http://<IP_DO_PROFESSOR>:1961`. Preencha o seu nome, o IP da sua máquina e o Token. Clique em **Conectar**.

3. Verificar a conexão

No seu terminal, rode:

```
sudo kubectl get nodes
```

(Agora você verá dois nós: o seu computador e o nó `worker-seu-nome` que o professor criou para você).

4. Deploy Distribuído (O Pulmão na Nuvem)

Vamos criar uma nova aplicação usando a imagem `nginxdemos/hello`. Ela é especial porque mostra o nome do Pod que respondeu à requisição. Além disso, usaremos uma regra de Anti-Affinity: *"Nunca coloque duas réplicas na mesma máquina"*.

Crie um arquivo chamado `app-nuvem.yaml`:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app-distribuida
spec:
  replicas: 2
  selector:
    matchLabels:
      app: distribuida
  template:
    metadata:
      labels:
        app: distribuida
    spec:
      affinity:
        podAntiAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            - labelSelector:
                matchExpressions:
```

```
                - key: app
                  operator: In
                  values: [distribuida]
            topologyKey: kubernetes.io/hostname
      containers:
        - name: web
          image: nginxdemos/hello:plain-text
          ports:
            - containerPort: 80
```

Aplique o arquivo e veja onde os pods caíram:

```
sudo kubectl apply -f app-nuvem.yaml
sudo kubectl get pods -o wide
```

(Você verá um pod rodando no nó local e outro rodando no nó `worker-seu-nome` – a nuvem).

5. O Teste de Fogo (Derrubando o Local)

Vamos provar que a nuvem segura a bronca. Vamos simular que o seu datacenter local caiu:

```
# 1. Descubra o nome do SEU no local
sudo kubectl get nodes

# 2. Isole o seu no (impeca novos pods de virem para ca)
sudo kubectl cordon <NOME_DO_SEU_NO>

# 3. Expulse os pods do seu no (eles vao para a nuvem)
sudo kubectl drain <NOME_DO_SEU_NO> --ignore-daemonsets --delete-emptydir-data
```

6. A Prova Final

Rode novamente:

```
sudo kubectl get pods -o wide
```

O pod que estava no seu nó foi morto e reagendado para o nó da nuvem (PC do professor). Você acaba de simular uma migração de nuvem!

Limpeza (Fim da Aula)

Para não sobrecarregar o PC do professor, apague a aplicação da nuvem e destrua o seu cluster local:

```
# Apaga o deployment e o service da nuvem
sudo kubectl delete deployment app-distribuida
sudo kubectl delete deployment meu-app

# Desinstala o k3s da sua maquina
/usr/local/bin/k3s-uninstall.sh
```